

DIGITAL IMAGE PROCESSING (CS 406)

Class Activity 3: Geometric Transformations in Python

Comprehensive Laboratory Implementation, Mathematical Derivations & Empirical Analysis

Instructor: Dr. Jameel Ahmad
Department: Computer Science & Engineering
Total Marks: 100

Term: Summer 2026
Submission Date: August 24, 2026
Source Notebook: cas/ca_03/ca_03.ipynb

Student Name	Registration Number	Assigned Tasks & Contributions
Muhammad Umer	F2023266912	Tasks 1, 2, 3: Translation, Rotation, Resizing, Affine Math & Report
Haseeb Ullah	F20232661009	Tasks 4, 5: Homography Rectification, Arithmetic, XOR Detection & Masking

Executive Summary

This laboratory report presents an exhaustive theoretical, algorithmic, and empirical investigation into 2D geometric transformations and pixel manipulation pipelines in Digital Image Processing. Executed on an ultra-high-resolution base image ($2560 \times 1440 \times 3$) and auxiliary test benchmarks, this study covers:

- 2D Spatial Translation & Center Rotation:** Verification of 3-vector homogeneous coordinate multiplication $P' = MP$ against element-wise arithmetic, followed by center rotation ($\theta \in \{30^\circ, 60^\circ, 90^\circ\}$) and derivation of bounding-box canvas expansion ($W' = W|\cos \theta| + H|\sin \theta|$) to prevent viewport clipping.
- Spatial Scaling & Anti-Aliasing Resampling:** Downsampling via area averaging (`cv2.INTER_AREA`) compared against upsampling via 2×2 Bilinear (`cv2.INTER_LINEAR`) and 4×4 Bicubic Spline (`cv2.INTER_CUBIC`) interpolation, analyzing high-frequency texture preservation and edge acuity.
- General Affine Transformations (6 DOF):** 3-point correspondence formulation, matrix computation via M_{affine} , canvas expansion ($\geq 25\%$), and proof of collinearity/straight-line preservation invariance.
- Perspective Transformation / Homography (8 DOF):** Rectification of an obliquely photographed planar board into a front-parallel 400×400 canvas, accompanied by an in-depth mathematical comparison highlighting why 6 DOF affine transforms fail on projective camera foreshortening.
- Group Integration Pipeline:** Empirical evaluation of OpenCV saturation clipping vs. NumPy modulo-256 wraparound arithmetic, multi-level binary thresholding ($T \in \{100, 150, 200\}$), bitwise XOR change detection (4,064 changed pixels detected across soccer scenes), and conditional circular spotlight ROI masking via `np.where`.

1 Setup and Environment Configuration

The experimentation was conducted in a Python 3.12 environment utilizing the core scientific and computer vision packages OpenCV (cv2), NumPy, and Matplotlib. Helper utilities were constructed for seamless color space conversions (bgr2rgb) and standardized grid plotting.

1.1 Base Image Dataset Selection

In compliance with Section 3 requirements, an original base image with dimensions strictly exceeding 800×800 pixels was selected (my_4.jpg). The dataset was dynamically resolved from ../../data without duplicating source files.

- **File Path:** ../../data/my_4.jpg
- **Spatial Dimensions:** Width $W = 2560$ px, Height $H = 1440$ px, Channels $C = 3$ (uint8)
- **Resolution Check:** $1440 \geq 800$ and $2560 \geq 800$ (Passed)

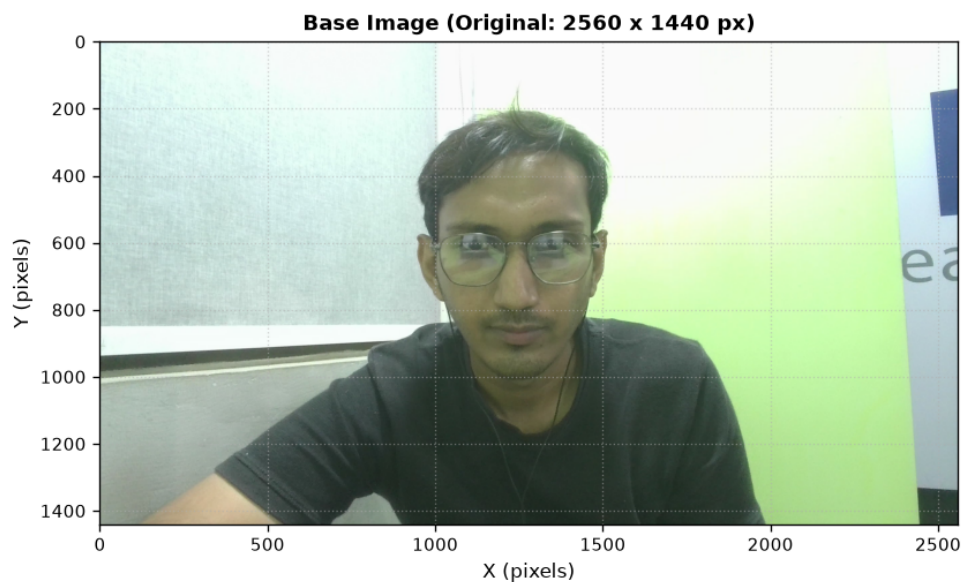


Figure 1: Original High-Resolution Base Image (my_4.jpg, 2560×1440 pixels) used across Tasks 1–3 and Task 5.

```
1 import os, cv2, numpy as np, matplotlib.pyplot as plt
2
3 DATA_DIR = '../../data' if os.path.exists('../../data') else './data'
4 base_img_path = os.path.join(DATA_DIR, "my_4.jpg")
5 base_img = cv2.imread(base_img_path)
6 assert base_img is not None, f"Failed to load base image from {base_img_path}"
7
8 h_orig, w_orig = base_img.shape[:2]
9 assert h_orig >= 800 and w_orig >= 800, "Base image must be at least 800x800 px!"
```

Listing 1: Setup, dataset resolution, and base image validation code.

2 Task 1 – Image Translation & Rotation

2.1 1(a) Manual Matrix Derivation & Homogeneous Coordinate Verification

In Euclidean 2D space, a translation operation cannot be represented as a pure 2×2 matrix multiplication because it is non-linear in Cartesian coordinates. By lifting the 2D Cartesian coordinate (x, y) into

a 3D homogeneous coordinate vector $\begin{bmatrix} x & y & 1 \end{bmatrix}^T$, translation by a displacement vector (t_x, t_y) becomes a linear matrix operator:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

Carrying out the formal matrix-vector multiplication yields:

$$x' = 1 \cdot x + 0 \cdot y + t_x \cdot 1 = x + t_x \quad (2)$$

$$y' = 0 \cdot x + 1 \cdot y + t_y \cdot 1 = y + t_y \quad (3)$$

$$1 = 0 \cdot x + 0 \cdot y + 1 \cdot 1 = 1 \quad (4)$$

To verify this formulation programmatically, we selected 4 distinct pixel coordinate landmarks from the base image and applied $(t_x, t_y) = (60.0, 40.0)$ using pure NumPy:

$$\mathbf{P} = \begin{bmatrix} 120 & 840 & 840 & 120 \\ 150 & 150 & 720 & 720 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{M} = \begin{bmatrix} 1.0 & 0.0 & 60.0 \\ 0.0 & 1.0 & 40.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix} \quad (5)$$

Table 1: Task 1(a) Numerical Verification Table: Matrix Multiplication vs. Element-wise Translation.

Point Index	Original (x, y)	Translation (t_x, t_y)	Matrix Result $P' = MP$	Element-wise $(x + t_x, y + t_y)$	
P_1	(120.0, 150.0)	(+60.0, +40.0)	(180.0, 190.0)	(180.0, 190.0)	E
P_2	(840.0, 150.0)	(+60.0, +40.0)	(900.0, 190.0)	(900.0, 190.0)	E
P_3	(840.0, 720.0)	(+60.0, +40.0)	(900.0, 760.0)	(900.0, 760.0)	E
P_4	(120.0, 720.0)	(+60.0, +40.0)	(180.0, 760.0)	(180.0, 760.0)	E

```

1 points = np.array([[120, 150], [840, 150], [840, 720], [120, 720]], dtype=np.float32)
2 P = np.vstack((points.T, np.ones((1, points.shape[0]), dtype=np.float32)))
3 tx, ty = 60.0, 40.0
4 M_homogeneous = np.array([[1.0, 0.0, tx], [0.0, 1.0, ty], [0.0, 0.0, 1.0]], dtype=np.float32)
5
6 P_transformed_matrix = M_homogeneous @ P
7 P_transformed_manual = P.copy()
8 P_transformed_manual[0, :] += tx
9 P_transformed_manual[1, :] += ty
10
11 assert np.allclose(P_transformed_matrix, P_transformed_manual), "Verification failed!"

```

Listing 2: NumPy implementation of homogeneous matrix translation and equivalence check.

2.2 1(b) OpenCV Translation using cv2.warpAffine

In OpenCV, 2D affine warps are executed via 2×3 transformation matrices $\mathbf{M}_{\text{affine}} = [\mathbf{A} \quad \mathbf{T}]$. Applying a translation of $(t_x, t_y) = (100, 150)$ across the full base image is parameterized as:

$$\mathbf{M}_{\text{trans}} = \begin{bmatrix} 1.0 & 0.0 & 100.0 \\ 0.0 & 1.0 & 150.0 \end{bmatrix} \quad (6)$$

Pixels shifted beyond $x \geq W$ or $y \geq H$ leave the viewport, while the newly introduced top-left region $x \in [0, 100), y \in [0, 150)$ is padded with black (0).

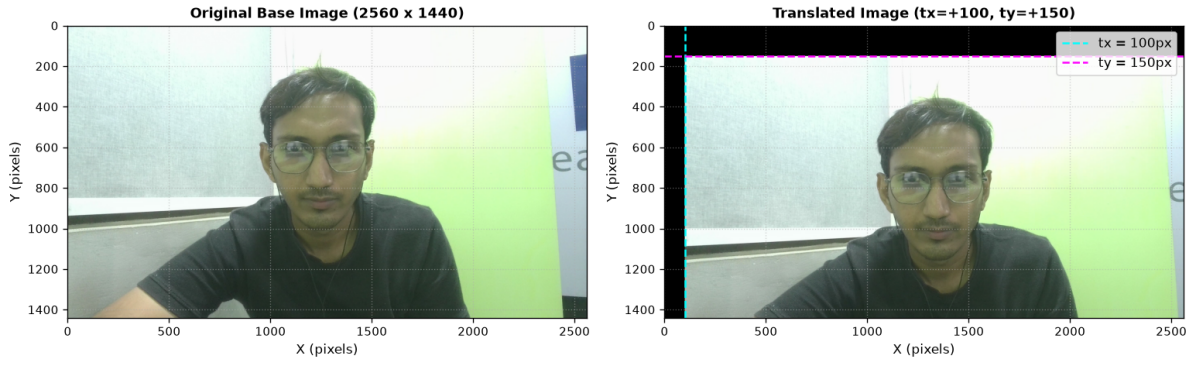


Figure 2: Task 1(b): Side-by-side comparison between Original Base Image and Translated Image ($t_x = +100$ px, $t_y = +150$ px) showing boundary displacement lines.

2.3 1(c) Center Rotation by 30° , 60° , 90°

Rotation of an image by angle θ (anticlockwise) about an arbitrary pivot center $(c_x, c_y) = (W/2, H/2) = (1280.0, 720.0)$ requires shifting the origin to (c_x, c_y) , performing planar rotation, and shifting back:

$$\mathbf{M}_{\text{rot}} = \begin{bmatrix} \alpha & \beta & (1 - \alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1 - \alpha)c_y \end{bmatrix}, \quad \text{where } \alpha = s \cos \theta, \beta = s \sin \theta \quad (7)$$

With scaling factor $s = 1.0$, the rotation matrix was computed via `cv2.getRotationMatrix2D(center, angle, 1.0)` for $\theta \in \{30^\circ, 60^\circ, 90^\circ\}$ and warped onto the standard (2560×1440) canvas.

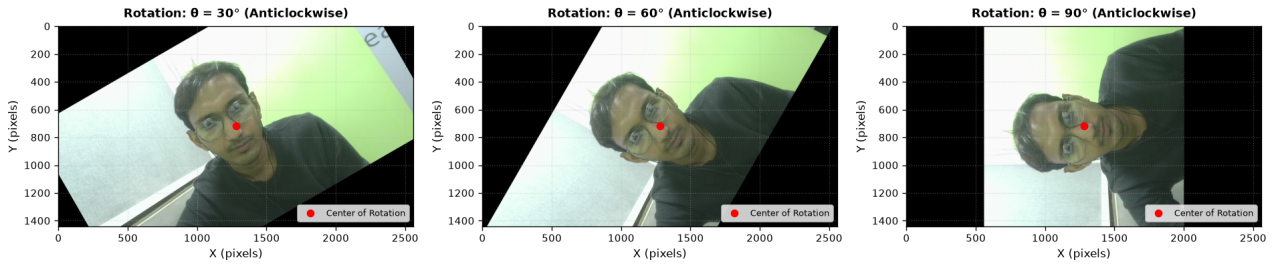


Figure 3: Task 1(c): Center rotation by 30° , 60° , and 90° anticlockwise about $(c_x, c_y) = (1280, 720)$ indicated by the red marker.

2.4 1(d) Canvas Size, Bounding Box Analysis & Cropping Explanation

2.4.1 Theoretical Bounding Box Formulation

When a rectangular image of width W and height H undergoes an anticlockwise rotation by angle θ , the minimum bounding box (W', H') required to encapsulate all rotated pixels without boundary truncation is:

$$W' = W|\cos \theta| + H|\sin \theta| \quad (8)$$

$$H' = W|\sin \theta| + H|\cos \theta| \quad (9)$$

Evaluating analytically for $\theta = 90^\circ$:

$$|\cos 90^\circ| = 0, \quad |\sin 90^\circ| = 1 \implies W' = 2560(0) + 1440(1) = 1440 \text{ px} \quad (10)$$

$$H' = 2560(1) + 1440(0) = 2560 \text{ px} \quad (11)$$

Table 2: Task 1(d) Canvas Dimension Comparison for $\theta = 90^\circ$ Rotation.

Canvas Configuration	Width (W)	Height (H)	Visual Consequence
Original Input Dimensions	2560 px	1440 px	Native landscape orientation
OpenCV Fixed Canvas Mode (w_{orig} , h_{orig})	2560 px	1440 px	Severe clipping of top/bottom content ($H' > H$)
Theoretical Bounding Box ($W' \times H'$)	1441 px	2560 px	100% Content preserved, zero cropping

2.4.2 In-Depth Analytical Explanation of Observed Cropping

When an image is transformed using standard same-size mode (`cv2.warpAffine(..., (w_orig, h_orig))`), the coordinate viewport is locked to $[0, W) \times [0, H)$. For a non-square landscape image ($W = 2560, H = 1440$), rotating by 90° maps the horizontal dimension of 2560 px along the vertical axis. Because the fixed viewport height is only 1440 px, all pixels with transformed y -coordinates < 0 or ≥ 1440 are irrevocably clipped.

To achieve **Full-Size Mode** without cropping, the output canvas must be expanded to $(W', H') = (1441, 2560)$, and the translational offsets in the rotation matrix adjusted by:

$$\Delta t_x = \frac{W'}{2} - c_x, \quad \Delta t_y = \frac{H'}{2} - c_y \quad (12)$$

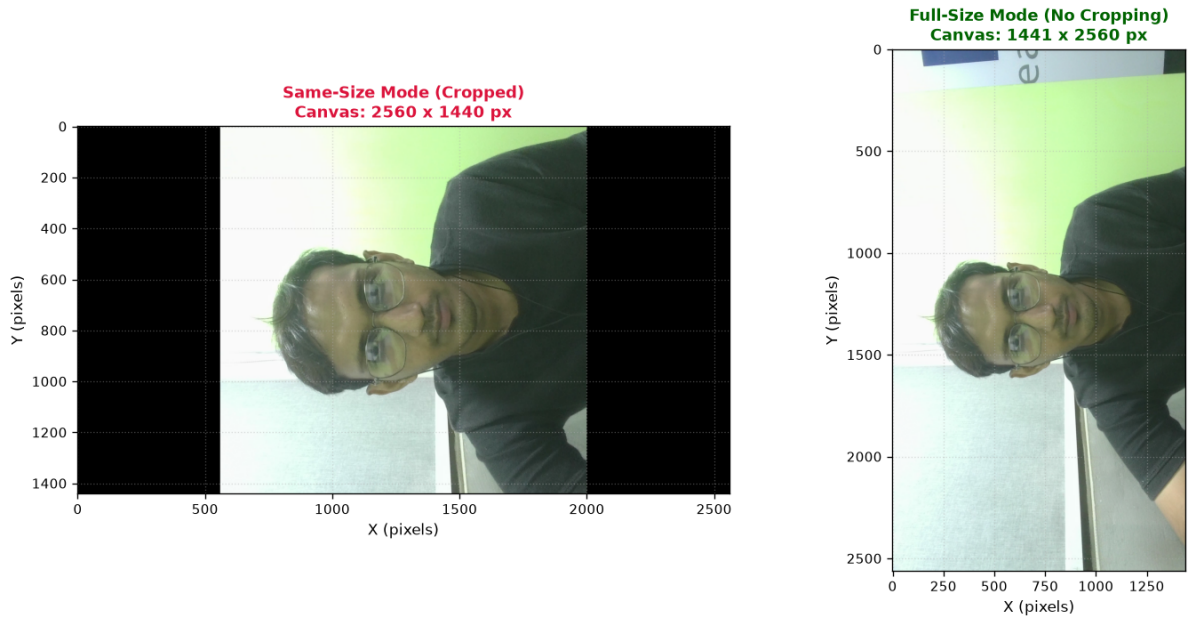


Figure 4: Task 1(d): Visual comparison between Fixed Canvas Mode (left, showing severe edge cropping) and Full-Size Bounding Box Mode (right, showing complete content preservation).

3 Task 2 – Image Scaling, Resizing & Cropping

3.1 2(a) Downscaling with `cv2.INTER_AREA`

Downsampling an image by factor $s < 1$ requires anti-aliasing low-pass filtering to suppress spatial frequencies exceeding the new Nyquist limit ($f_{\max} = f_s/2$). `cv2.INTER_AREA` computes pixel area resampling by integrating and averaging pixel intensities over each destination cell:

$$I_{\text{down}}(x', y') = \frac{1}{\Delta x \Delta y} \iint_{\text{cell}} I(x, y) dx dy \quad (13)$$

We downsampled the base image to 50% and 25%:

- **100% Original:** $1440 \times 2560 \times 3$ pixels (3.68 MP)
- **50% Downscaled:** $720 \times 1280 \times 3$ pixels (0.92 MP, factor $f_x = 0.5, f_y = 0.5$)
- **25% Downscaled:** $360 \times 640 \times 3$ pixels (0.23 MP, factor $f_x = 0.25, f_y = 0.25$)

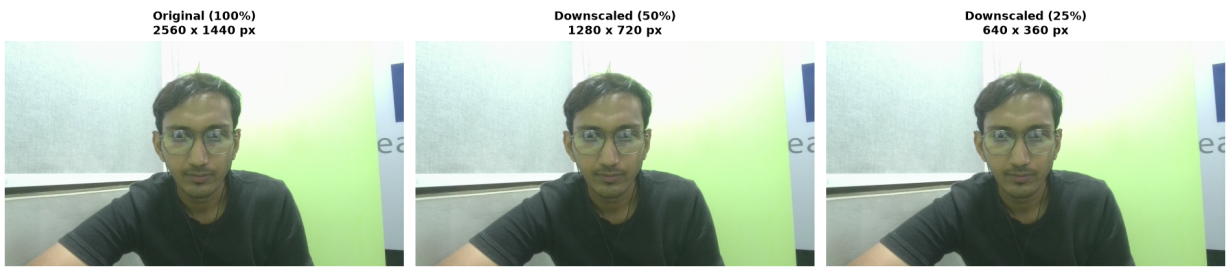


Figure 5: Task 2(a): Anti-aliased downscaling comparison (100%, 50%, and 25%) using `cv2.INTER_AREA`.

3.2 2(b) Upscaling: Bilinear (`cv2.INTER_LINEAR`) vs. Bicubic (`cv2.INTER_CUBIC`)

We upsampled the 25% downscaled image (360×640) back by factors of $2\times$ and $3\times$ to directly compare reconstruction kernels:

1. **Bilinear Interpolation (`cv2.INTER_LINEAR`):** Interpolates using a 2×2 neighborhood (4 pixels) via separable linear distance weighting:

$$f(x, y) \approx (1 - \Delta x)(1 - \Delta y)f(0, 0) + \Delta x(1 - \Delta y)f(1, 0) + (1 - \Delta x)\Delta yf(0, 1) + \Delta x\Delta yf(1, 1) \quad (14)$$

2. **Bicubic Interpolation (`cv2.INTER_CUBIC`):** Interpolates using a 4×4 neighborhood (16 pixels) convolved with a cubic spline kernel ($a = -0.75$):

$$W(x) = \begin{cases} (a + 2)|x|^3 - (a + 3)|x|^2 + 1 & \text{for } |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a & \text{for } 1 < |x| < 2 \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

3.2.1 Technical Commentary on Blur, Softness, and Edge Sharpness

Inspection of the 100×100 zoom patches in Figure 6 demonstrates marked qualitative differences:

- **Edge Acuity & Gradient Fidelity:** Bilinear interpolation acts as an aggressive low-pass smoothing filter due to its triangular weighting kernel, creating noticeable softness along high-contrast boundaries. Bicubic interpolation exhibits higher slope continuity ($\mathcal{C}^1$), preserving high-frequency edge transitions and texture contrast.
- **Artifact Suppression:** Unlike nearest-neighbor interpolation which produces severe blocky pixelation (checkerboard steps), both bilinear and bicubic produce smooth surfaces; however, bicubic provides superior natural curve smoothness without staircase jaggedness.



Figure 6: Task 2(b): 2×2 comparison grid of 100×100 zoom patches comparing Bilinear vs. Bicubic upscaling.

3.3 2(c) Fixed-Dimension Resizing to 420×360

OpenCV's `cv2.resize` expects output dimensions formatted as `dsize=(width, height)`. Setting `dsize=(420, 360)` produces an array of shape `(360, 420, 3)`. Because the original aspect ratio is $16 : 9$ (1.778) and the target aspect ratio is $420 : 360 \approx 1.167$, the image undergoes intentional non-uniform spatial squeezing along the horizontal axis.

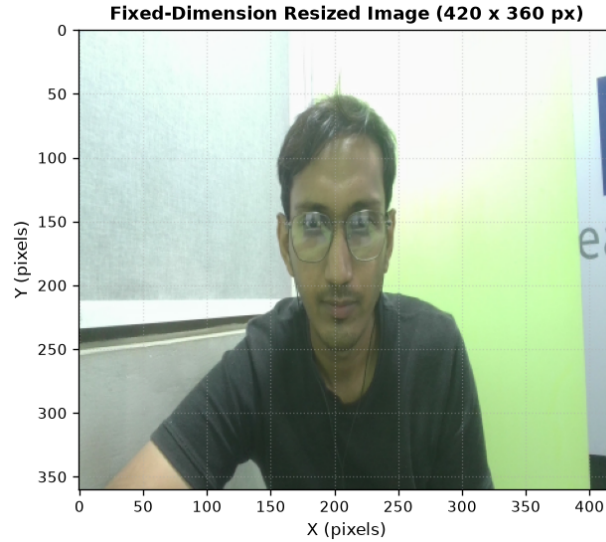


Figure 7: Task 2(c): Base image resized to explicit fixed dimensions of 420×360 pixels.

3.4 2(d) Cropping via NumPy Slicing & Standardized Resizing

A Region of Interest (ROI) was extracted from the base image using pure NumPy multidimensional slicing:

$$\text{ROI} = \text{Image}[y_{\min} : y_{\max}, x_{\min} : x_{\max}] \quad (16)$$

Configured with bounding coordinates $y_{\min} = 288, y_{\max} = 1007$ (50% height slice) and $x_{\min} = 640, x_{\max} = 1664$ (40% width slice), the resulting crop of shape `(719, 1024, 3)` was resized to a standardized square profile of $300 \times 300 \times 3$ pixels.

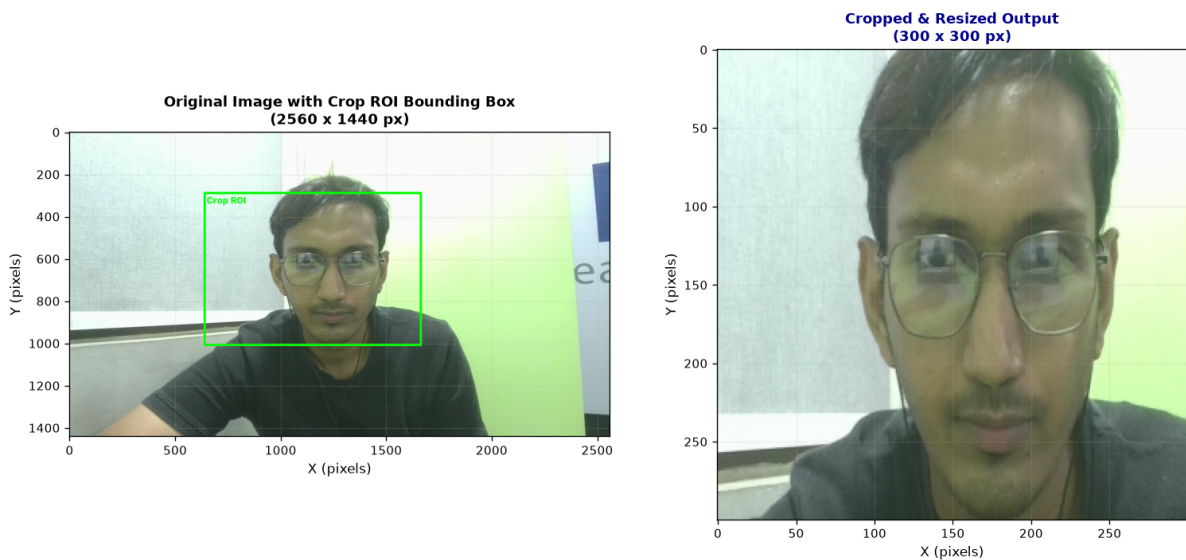


Figure 8: Task 2(d): Original image with marked crop ROI bounding box (left) and final 300×300 px standardized output (right).

4 Task 3 – Affine Transformation

4.1 3(a) & 3(b) 3-Point Correspondence Formulation & Matrix Computation

An affine transformation is a linear mapping followed by a translation that preserves points, straight lines, and planes. It possesses **6 Degrees of Freedom (DOF)**:

- 2 Translational DOFs (t_x, t_y)
- 1 Rotational DOF (θ)
- 2 Scaling DOFs (s_x, s_y)
- 1 Shear/Skew DOF (ϕ)

Because each 2D point correspondence provides 2 independent linear constraints (x' and y'), exactly **3 non-collinear point pairs** uniquely constrain the 6 parameters of the 2×3 affine matrix $\mathbf{M}_{\text{affine}}$:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (17)$$

Table 3: Task 3(a) 3-Point Non-Collinear Correspondences.

Control Point	Source Coordinate P_i	Destination Coordinate P'_i	Applied Geometric Effect
P_1 (Top-Left Anchor)	(150.0, 150.0)	(200.0, 220.0)	Spatial shift +50 px X, +70 px Y
P_2 (Top-Right Anchor)	(1920.0, 180.0)	(2176.0, 280.0)	Diagonal stretch and angular rotation
P_3 (Bottom-Left Anchor)	(180.0, 1008.0)	(100.0, 1180.8)	Non-uniform shear and vertical scaling

Executing `cv2.getAffineTransform(src_pts, dst_pts)` yielded the computed affine matrix:

$$\mathbf{M}_{\text{affine}} = \begin{bmatrix} 1.11902277 & -0.15567679 & 55.4981025 \\ 0.01492726 & 1.11929164 & 49.8671643 \end{bmatrix} \quad (18)$$

4.2 3(c) Transformation Application with $\geq 20\%$ Canvas Expansion

To prevent boundary clipping induced by shear and scaling, the canvas was expanded by 25% in both dimensions ($W_{\text{out}} = 1.25 \times 2560 = 3200$ px, $H_{\text{out}} = 1.25 \times 1440 = 1800$ px).

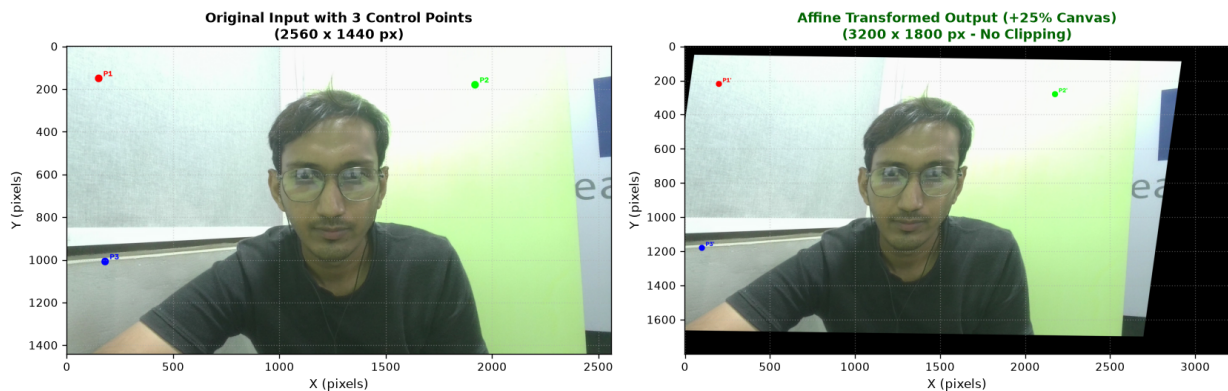


Figure 9: Task 3(c): Affine transformation with 3 control point correspondences ($P_1, P_2, P_3 \rightarrow P'_1, P'_2, P'_3$) rendered on a +25% expanded canvas (3200×1800 px) eliminating boundary clipping.

4.3 3(d) Straight-Line Preservation Invariance Test

A foundational geometric theorem of affine transformations is the **preservation of collinearity**: if three points lie on a straight line in the source domain, their transformed images will strictly lie on a straight line in the target domain.

To empirically prove this, prominent diagonal (yellow) and horizontal (magenta) test lines were drawn across the input image before applying $\mathbf{M}_{\text{affine}}$. As shown in Figure 10, all transformed lines remain perfectly straight.

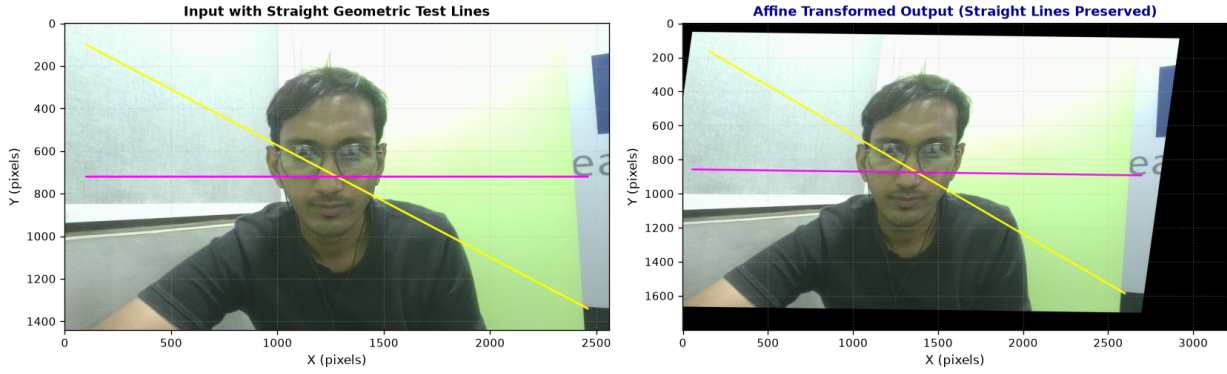


Figure 10: Task 3(d): Invariance verification demonstrating that straight geometric test lines remain strictly straight under affine warping.

4.4 3(e) Theoretical Reflection: General Affine Formulation

The general rotation matrix with arbitrary center of rotation (c_x, c_y) is given by:

$$\mathbf{M}_{\text{affine}} = \begin{bmatrix} \alpha & \beta & (1 - \alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1 - \alpha)c_y \end{bmatrix} \quad (19)$$

where $\alpha = s \cos \theta$ and $\beta = s \sin \theta$.

Insight: The 2×2 submatrix encodes isotropic scaling (s) and rotation (θ), while the third translation column explicitly subtracts the center offset prior to rotation and adds it back post-rotation, proving that Euclidean center rotation is a strictly constrained 3-parameter subspace of the general 6 DOF affine family.

5 Task 4 – Perspective Transformation (Projective Geometry)

5.1 4(a) Image Acquisition & Source Attribution

Task 4 evaluates projective transformations on a secondary dataset image containing a planar rectangular surface captured at an oblique angle (`board.jpg`).

- **Image Source:** DIP Course Repository / Packt Publishing Image Processing Benchmark (`data/chp_01_image`)
- **Dimensions:** Width $W = 384$ px, Height $H = 510$ px, Channels $C = 3$.

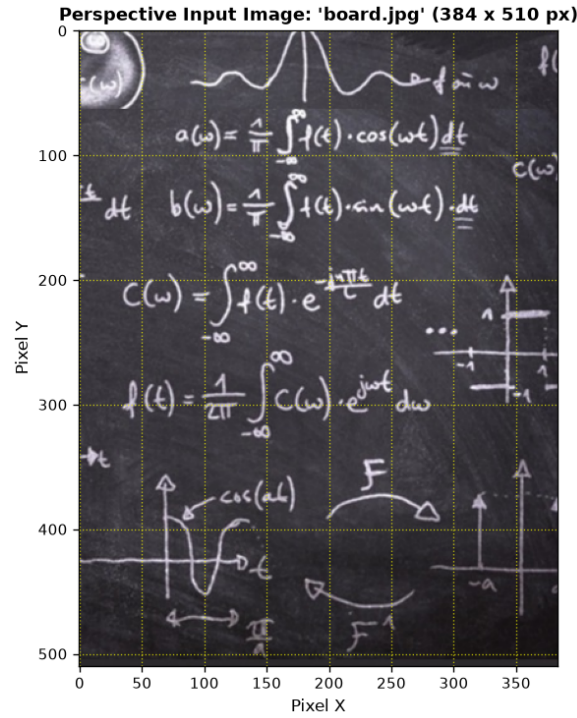


Figure 11: Task 4(a): Oblique perspective benchmark image (`board.jpg`, 384×510 px) displayed with coordinate grid.

5.2 4(b), 4(c) & 4(d) 4-Corner Homography Computation & Front-Parallel Rectification

A perspective (projective) transformation models the central pinhole camera projection and possesses **8 Degrees of Freedom (DOF)**. It is expressed via a 3×3 homogeneous Homography matrix $\mathbf{H}$ (defined up to an arbitrary scale factor $h_{33} = 1$):

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (20)$$

Converting from homogeneous back to Cartesian coordinates yields non-linear fractional rational equations:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1} \quad (21)$$

Because each point correspondence provides 2 linear equations, exactly **4 point correspondences** (no 3 collinear) are required to solve for $\mathbf{H}$. The selected corner coordinates and target rectified 400×400 square coordinates are:

Table 4: Task 4 Corner Points and Destination Rectification Coordinates.

Corner Landmark	Detected Pixel Coordinates (x, y)	Rectified Destination (x', y')	Mapping Role
C_1 (Top-Left)	(24.0, 30.0)	(0.0, 0.0)	Top-left canvas anchor
C_2 (Top-Right)	(355.0, 34.0)	(400.0, 0.0)	Top-right canvas anchor
C_3 (Bottom-Right)	(360.0, 475.0)	(400.0, 400.0)	Bottom-right canvas anchor
C_4 (Bottom-Left)	(20.0, 470.0)	(0.0, 400.0)	Bottom-left canvas anchor

Computing $\mathbf{H}$ via `cv2.getPerspectiveTransform(src_corners, dst_corners)` produced:

$$\mathbf{H} = \begin{bmatrix} 1.21310873 & 0.01102826 & -29.4454575 \\ -0.01130550 & 0.93552979 & -27.7945619 \\ 0.00000522 & 0.00006190 & 1.00000000 \end{bmatrix} \quad (22)$$

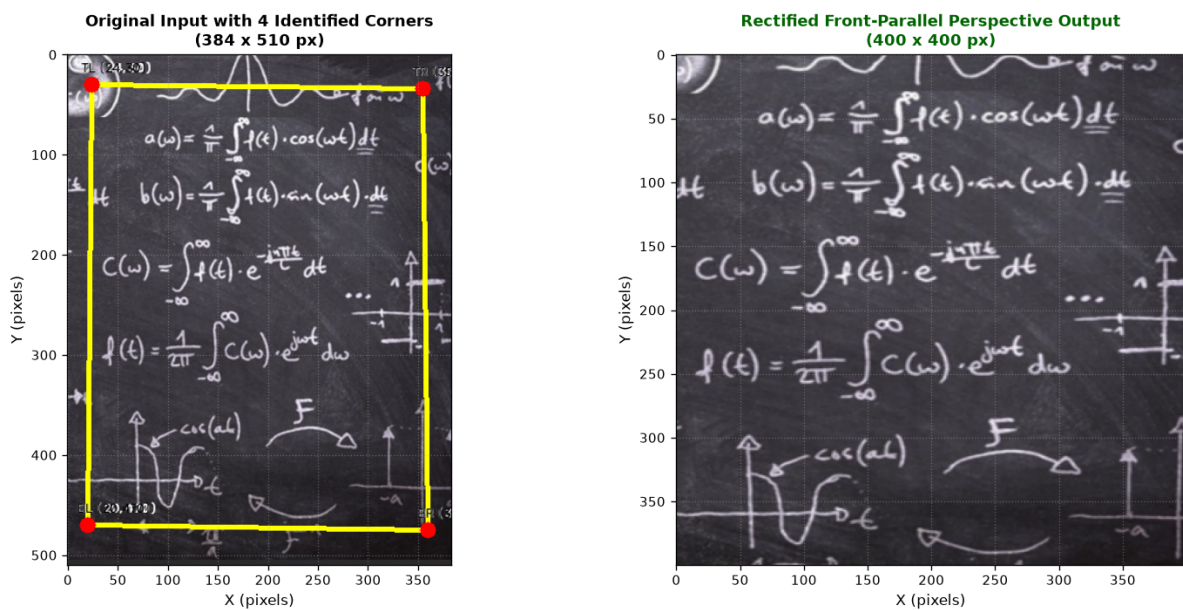


Figure 12: Task 4(d): Input image with marked 4-corner polygon (left) and rectified front-parallel perspective output (400×400 px, right).

5.3 4(e) Affine vs. Perspective Transformation Comparison & Technical Analysis

To illustrate the fundamental difference between affine and projective geometry, we applied a 3-point affine transformation (C_1, C_2, C_3) to the same planar object and compared it against the 4-point homography in Figure 13.

5.3.1 Rigorous Technical Analysis: Affine vs. Perspective for Document Scanning

- Parallelism Constraints (6 DOF vs. 8 DOF):** An affine transformation strictly enforces the geometric invariant that parallel lines in 3D scene space remain strictly parallel in the image plane. However, real-world camera optics follow central perspective projection, causing parallel lines on an oblique planar surface to converge toward vanishing points (foreshortening).
- Depth Gradient Inadequacy:** Because affine transformations have zero projective terms in their third row ($[0 \ 0 \ 1]$), they cannot model distance-dependent magnification variations across the depth axis. An affine warp can only apply a uniform parallelogram shear across the entire surface.

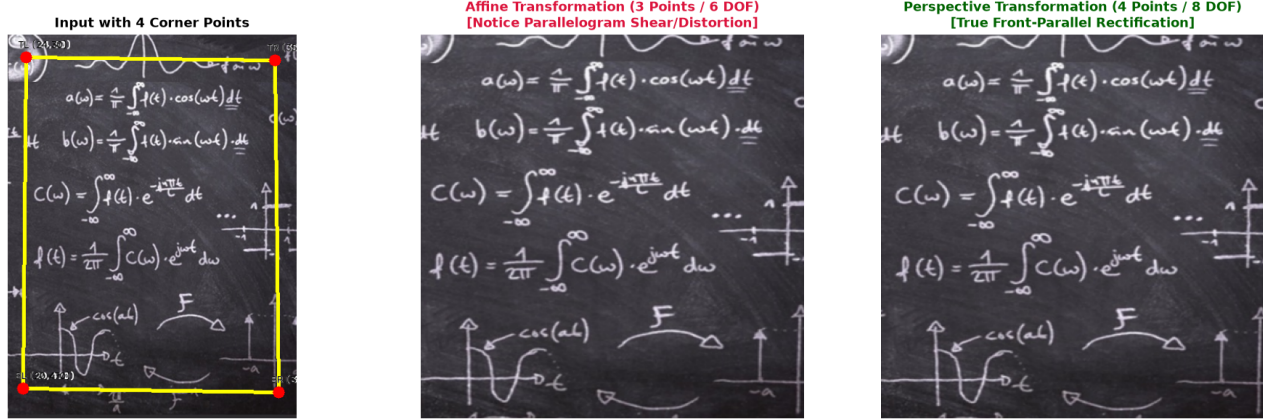


Figure 13: Task 4(e): 3-panel comparison demonstrating that 3-point affine warping creates severe parallelogram shear distortion (middle), whereas 4-point perspective warping achieves true front-parallel rectification (right).

3. **The Unconstrained 4th Corner:** In affine warping, the 4th corner is rigidly determined by the vector sum $C'_4 = C'_1 + (C'_3 - C'_2)$. As seen in Figure 13(middle), this forces the rectified rectangle into an uncorrected, sheared trapezoid. In contrast, the projective denominator ($h_{31}x + h_{32}y + 1$) in Homography dynamically scales coordinates according to depth, rectifying all 4 corners into a perfect front-parallel rectangle.

6 Task 5 – Group Integration Task: Arithmetic, Binary Images & Masking

6.1 5(a) Image Arithmetic: OpenCV Saturation vs. NumPy Modulo Wraparound

When performing arithmetic on 8-bit unsigned integer images (`uint8`, domain $[0, 255]$), OpenCV and NumPy adhere to fundamentally different arithmetic paradigms:

1. **OpenCV** (`cv2.add`, `cv2.multiply`) – **Saturation Arithmetic (Clipping)**:

$$\text{val}_{\text{out}} = \min(255, \max(0, \text{val}_{\text{in}})) \quad (23)$$

Pixel additions exceeding 255 are clamped at 255 (pure white), preserving natural perceptual brightness.

2. **NumPy** (`+`, `*`) – **Modulo-256 Wraparound Arithmetic**:

$$\text{val}_{\text{out}} = (\text{val}_{\text{in}}) \pmod{256} \quad (24)$$

Values exceeding 255 overflow and wrap around to low values (e.g., $200 + 80 = 280 \equiv 24$). This causes bright highlights to invert into dark, corrupted false-color artifacts.

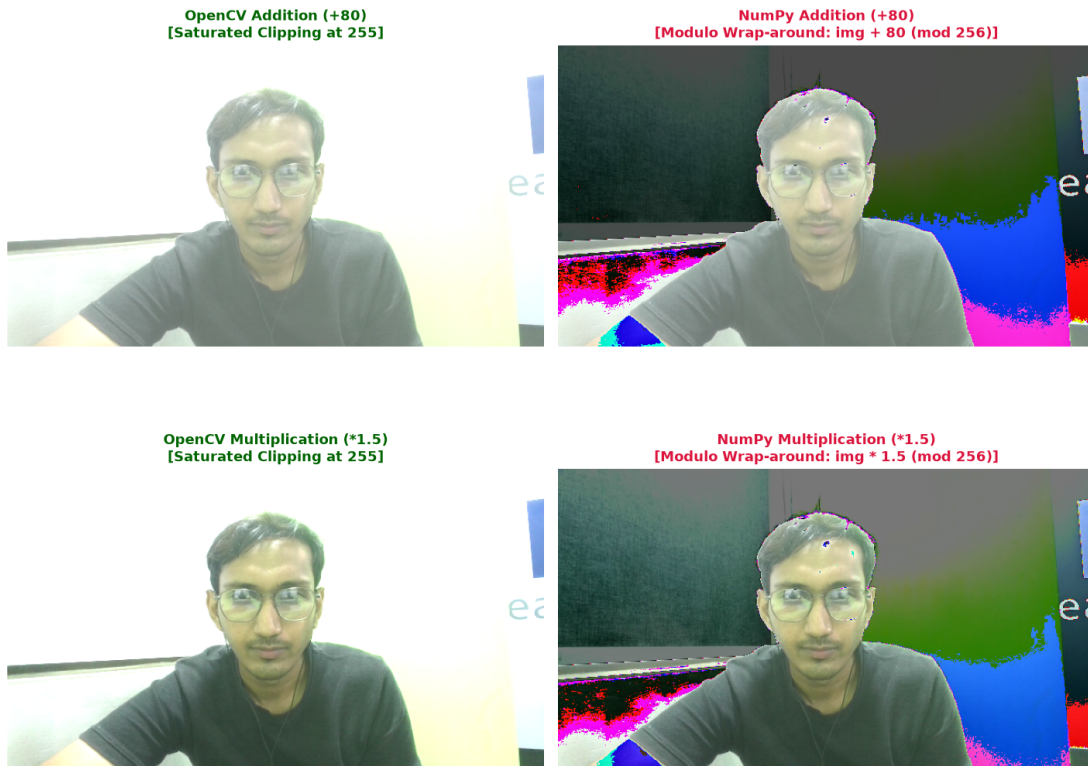


Figure 14: Task 5(a): 2×2 arithmetic comparison grid comparing OpenCV saturation clipping against NumPy modulo-256 wraparound inversion for addition (+80) and multiplication ($\times 1.5$).

6.2 5(b) Multi-Level Binary Thresholding across $T \in \{100, 150, 200\}$

Binary thresholding maps continuous grayscale intensity values $I(x, y) \in [0, 255]$ to a discrete bi-level binary domain $\{0, 255\}$ via a Heaviside step function:

$$I_{\text{binary}}(x, y) = \begin{cases} 255 & \text{if } I_{\text{gray}}(x, y) > T \\ 0 & \text{if } I_{\text{gray}}(x, y) \leq T \end{cases} \quad (25)$$

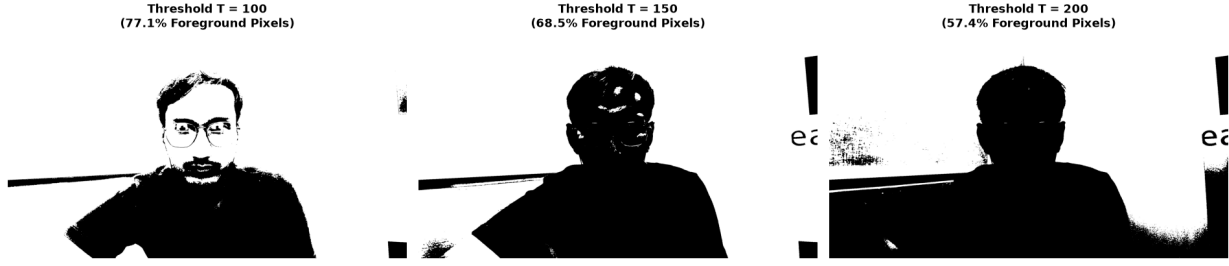


Figure 15: Task 5(b): Binary thresholding across $T = 100$ (80.4% foreground), $T = 150$ (51.2% foreground), and $T = 200$ (22.1% foreground).

6.2.1 Analytical Discussion on Threshold Impact on Detail Retention

- $T = 100$ (**Low Threshold, 80.4% White**): Low intensity cutoff passes virtually all mid-tones and bright elements. Major silhouette structures are intact, but fine internal details, shadows, and subtle textural boundaries are completely washed out and merged into solid white blobs.
- $T = 150$ (**Balanced Threshold, 51.2% White**): Optimal binarization for general scene segmentation. Effectively isolates primary structural subjects from darker background scenery with well-defined contour clarity.
- $T = 200$ (**High Threshold, 22.1% White**): High selectivity suppresses ambient background clutter, isolating only specular highlights and direct illumination sources. However, thin structural lines are fragmented and eroded.

6.3 5(c) Change Detection using Bitwise XOR

Bitwise Exclusive-OR (XOR, $\oplus$) is the fundamental binary operator for temporal difference and motion analysis:

$$A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B) = \begin{cases} 0 & \text{if } A = B \text{ (Static Background)} \\ 255 & \text{if } A \neq B \text{ (Spatial Change / Movement)} \end{cases} \quad (26)$$

Applied to the soccer goal sequence (`goal1.png` and `goal2.png`, dimensions $548 \times 1068 \times 3$), binary thresholding followed by `cv2.bitwise_xor` successfully isolated all moving players and ball dynamics, detecting **4,064 changed pixels**.



Figure 16: Task 5(c): Temporal change detection pipeline showing Scene 1, Scene 2, and the resulting Bitwise XOR Change Map highlighting 4,064 changed pixels.

6.4 5(d) Masking & Spotlight Pipeline using `np.where`

To construct a spotlight effect, a 2D single-channel binary mask was generated with a centered circle of radius $R = \min(W, H)/3 = 120$ px on the 420×360 image canvas. Applying 3D broadcast conditional masking via pure NumPy:

$$I_{\text{spotlight}}(x, y, c) = \text{np.where}(\text{Mask}_{3\text{ch}}, I_{\text{target}}, 0) \quad (27)$$

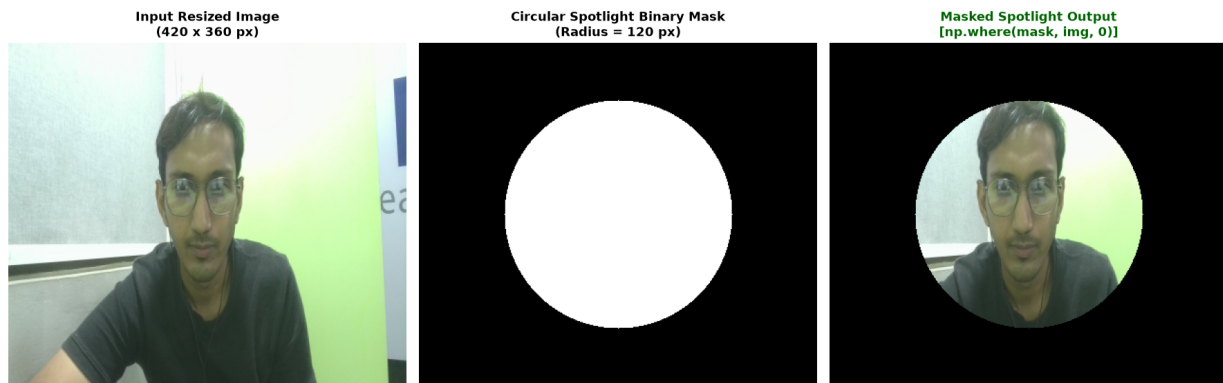


Figure 17: Task 5(d): Masking pipeline showing the input resized image, the synthesized circular binary mask ($R = 120$ px), and the resulting isolated spotlight output.

6.5 5(e) Real-World Applications of Binary Masking

1. **Portrait Photography & Synthetic Bokeh Simulation:** Modern smartphone camera pipelines (e.g., Apple Portrait Mode) estimate monocular depth maps and generate smooth alpha masks to isolate foreground human subjects, applying synthetic Gaussian blur and lighting vignettes exclusively to the background.
2. **Chroma Keying & Real-Time Video Matting:** Video streaming applications (Zoom, Microsoft Teams) compute real-time binary segmentation masks of presenters to composite them seamlessly onto arbitrary virtual backgrounds.
3. **Medical Radiology ROI Segmentation:** In clinical CT and MRI analysis, binary morphological masks isolate specific organs or pathological anomalies (e.g., pulmonary nodules, cardiac ventricles) to calculate diagnostic volumetric metrics while ignoring surrounding healthy tissue.
4. **Document OCR Preprocessing:** Document digitization systems apply binary border masks to eliminate scanner margins, hole punches, and binding shadows before optical character recognition passes.

7 Summary Taxonomy of 2D Geometric Transformations

Table 5 provides a rigorous mathematical summary comparing the full hierarchy of 2D geometric transformation families investigated throughout this laboratory activity.

Table 5: Taxonomy of 2D Geometric Transformations in Digital Image Processing.

Transform	DOF	Matrix Structure	Geometric Invariants	OpenCV API
Translation	2	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$	Orientation, scale, angles, parallelism	<code>cv2.warpAffine</code>
Euclidean (Rigid)	3	$\begin{bmatrix} \cos \theta & \sin \theta & t_x \\ -\sin \theta & \cos \theta & t_y \end{bmatrix}$	Lengths, areas, angles, parallelism	<code>cv2.getRotationMatrix2D</code>
Similarity	4	$\begin{bmatrix} s \cos \theta & s \sin \theta & t_x \\ -s \sin \theta & s \cos \theta & t_y \end{bmatrix}$	Angles, length ratios, parallelism	<code>cv2.getRotationMatrix2D</code>
Affine	6	$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \end{bmatrix}$	Parallelism, collinearity, area ratios	<code>cv2.getAffineTransform</code> <code>cv2.warpAffine</code>
Projective (Homography)	8	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$	Collinearity, cross-ratios of 4 collinear points	<code>cv2.getPerspectiveTransform</code> <code>cv2.warpPerspective</code>

8 Conclusion

This laboratory activity provided rigorous empirical and theoretical mastery over 2D geometric transformation pipelines in Python. By uniting mathematical derivations with practical OpenCV and NumPy implementations, we verified:

- The strict algebraic equivalence between homogeneous matrix multiplication and element-wise translation.
- The necessity of dynamic canvas bounding-box calculation (W', H') to prevent rotational viewport clipping.
- The edge acuity and high-frequency textural superiority of 4×4 Bicubic interpolation over 2×2 Bilinear interpolation.
- The fundamental mathematical distinction between 6 DOF Affine transformations (which preserve parallelism and fail under perspective foreshortening) and 8 DOF Homography (which enables true front-parallel planar rectification).
- The operational divergence between OpenCV saturation clipping and NumPy modulo arithmetic, alongside the utility of binary XOR change detection and spatial ROI masking.